System Design

chat Application → whatsapp

↳ HLD → high level Design

↳ send → messages
  ↳ individuals
  ↳ groups ✓

↳ sending media
  ↳ size issue ✓

↳ acknowlegment
  ↳ single tick
  ↳ grey double tick
  ↳ Blue double tick

↳ receive msgs
  ↳ individual
  ↳ group

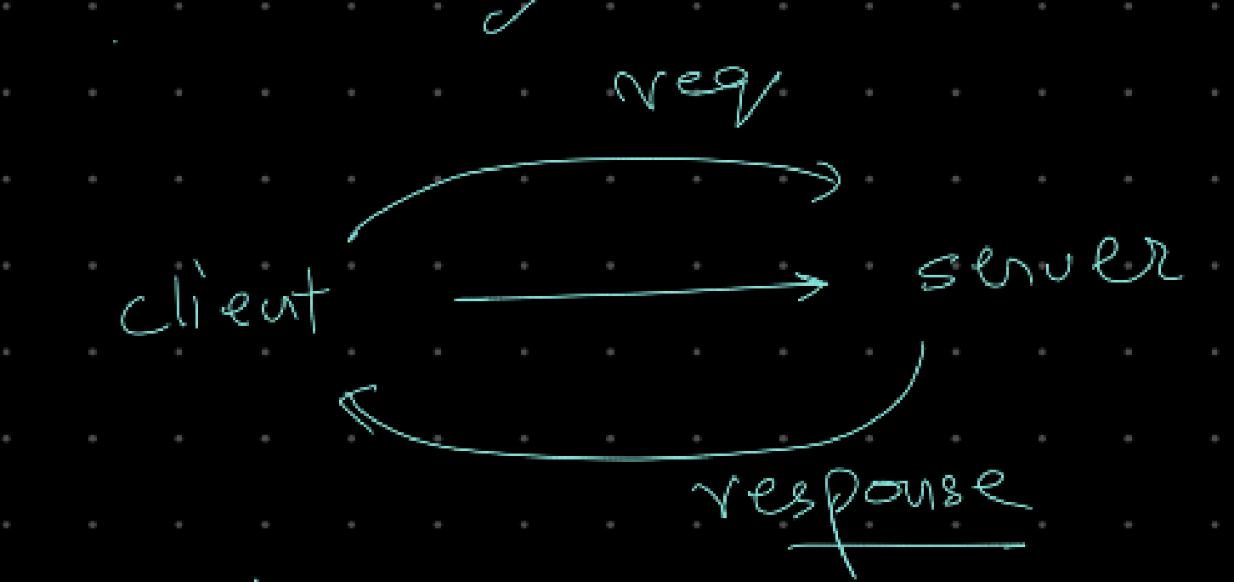
↳ storage → 30 days.

↳ delivey even when
  ↳ offline

↳ [multi - device support]

} Functional Requirements

↳ Presence Indicator
  ↳ online / not
  ↳ typing


req
client
server
response

Non functional Requirements

① ↳ Latecny sensitive
  ↳ fast system
↳ msg should ⇒ $\boxed{500\ ms}$

t2 - t1 ⇒ Latency

(msg delivey) → 500 ms

② Guarantee msg order → gaurantee persistence → storage

③ 1→2 Billion users → 100 → 1000 msg per day ⇒ 100M x 100 ⇒ (1B)

No. of write → Billions ✓

① sql
② NO SQL

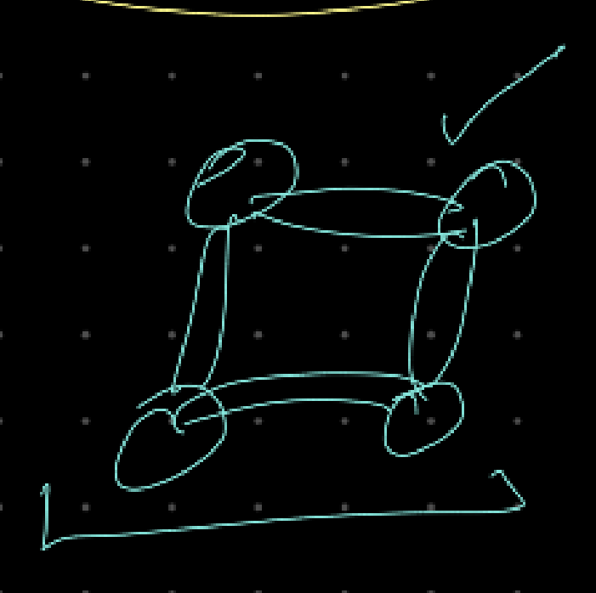
sql → ACID → high write (X)

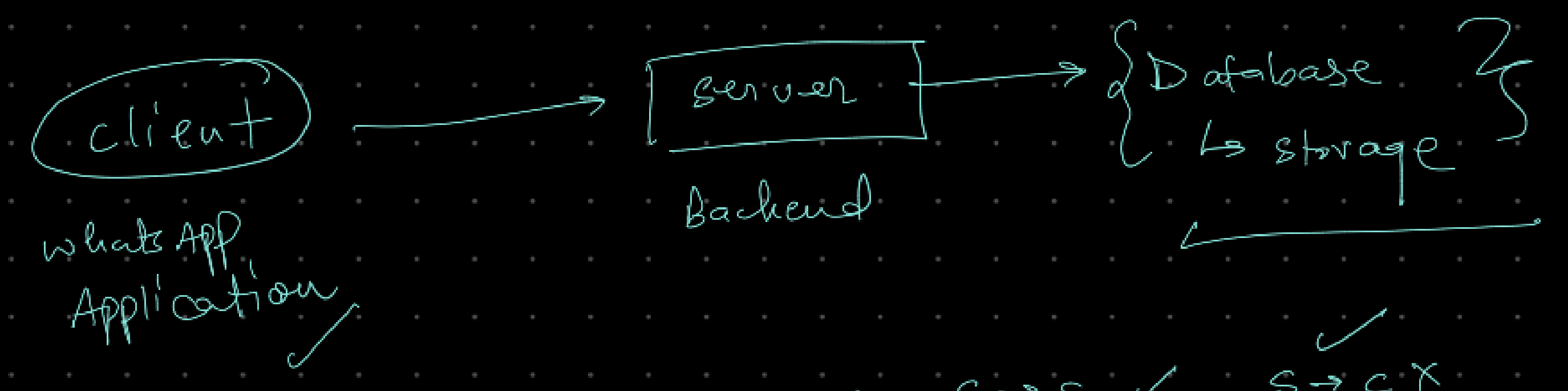
Nosql → ACID → high write ✓ → Nosql → (Dynamo DB)

④ (C A P) → Consistency [Availability] ✓

⇓ Partition Tolerance

⇓ highly available and eventual Consistency.




client
whats App Application
server
Backend
{Database ↳ storage}

http ↳ client → server

{client → s} → A

{c → s, s → c} → B ✓

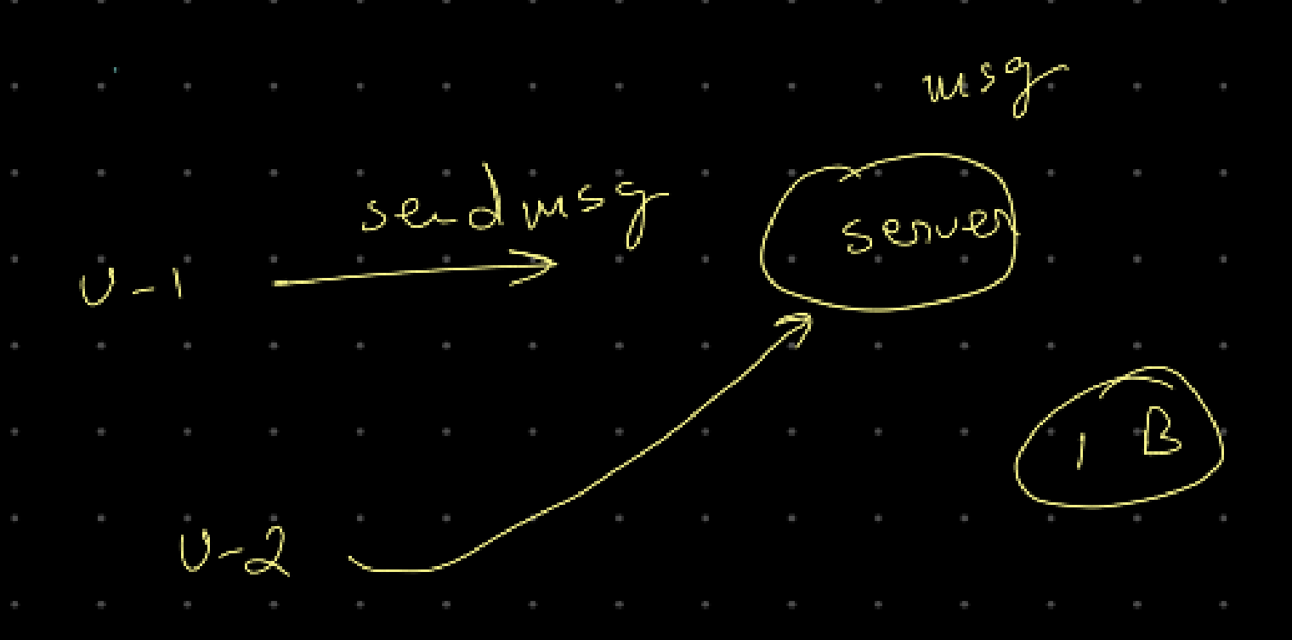
http → c → s ✓ s → c X

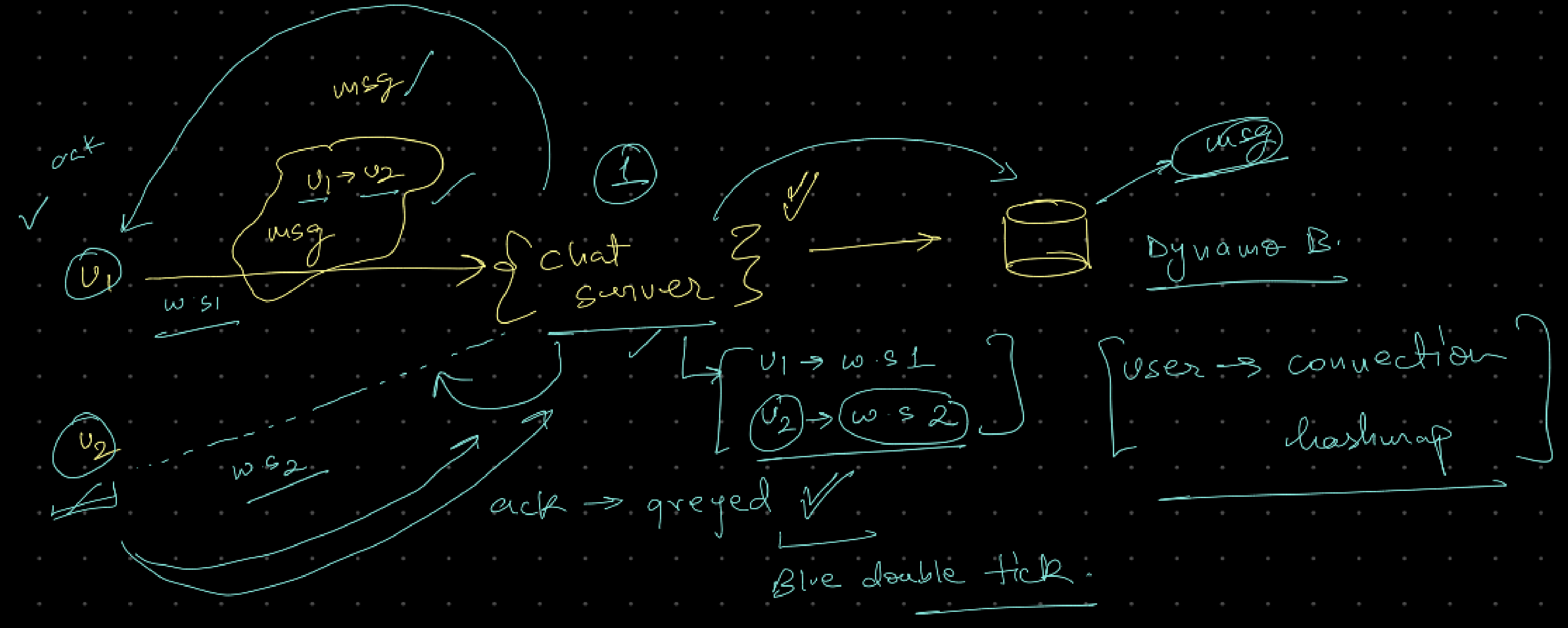
SSE → s → c ✓

[websockets → c → s ✓ s → c ✓] ✓ } ✓

webRTC → c → s ✓ s → c ✓] ✓
↳ calling, video calling

} over-kill


msg
U-1
sendmsg
server
U-2
1 B


ack
msg
U1 → U2
msg
①
chat server
U1
w.s1
U2
ws2
Dynamo B.
msg
U1 → w.s1
U2 → w.s2
[user → connection hashmap]
ack → greyed ✓
Blue double tick.

→ msg → Reciever → (U2) → (ws2)

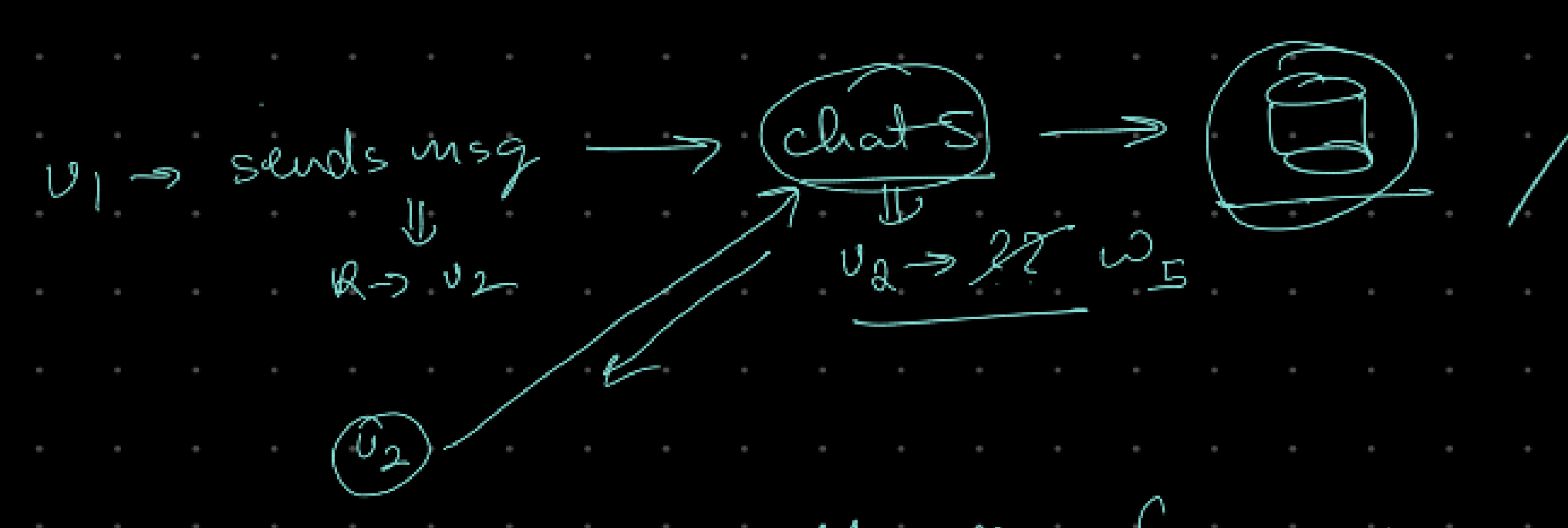
[U1 → ws1
(U2) → ws2]

Each connection is unique

U1 → ws1
U2 → ws2
U3 → ws3

U1 → ~~ws1~~
U1 →

U1 → ws1
U1 → [ws100]

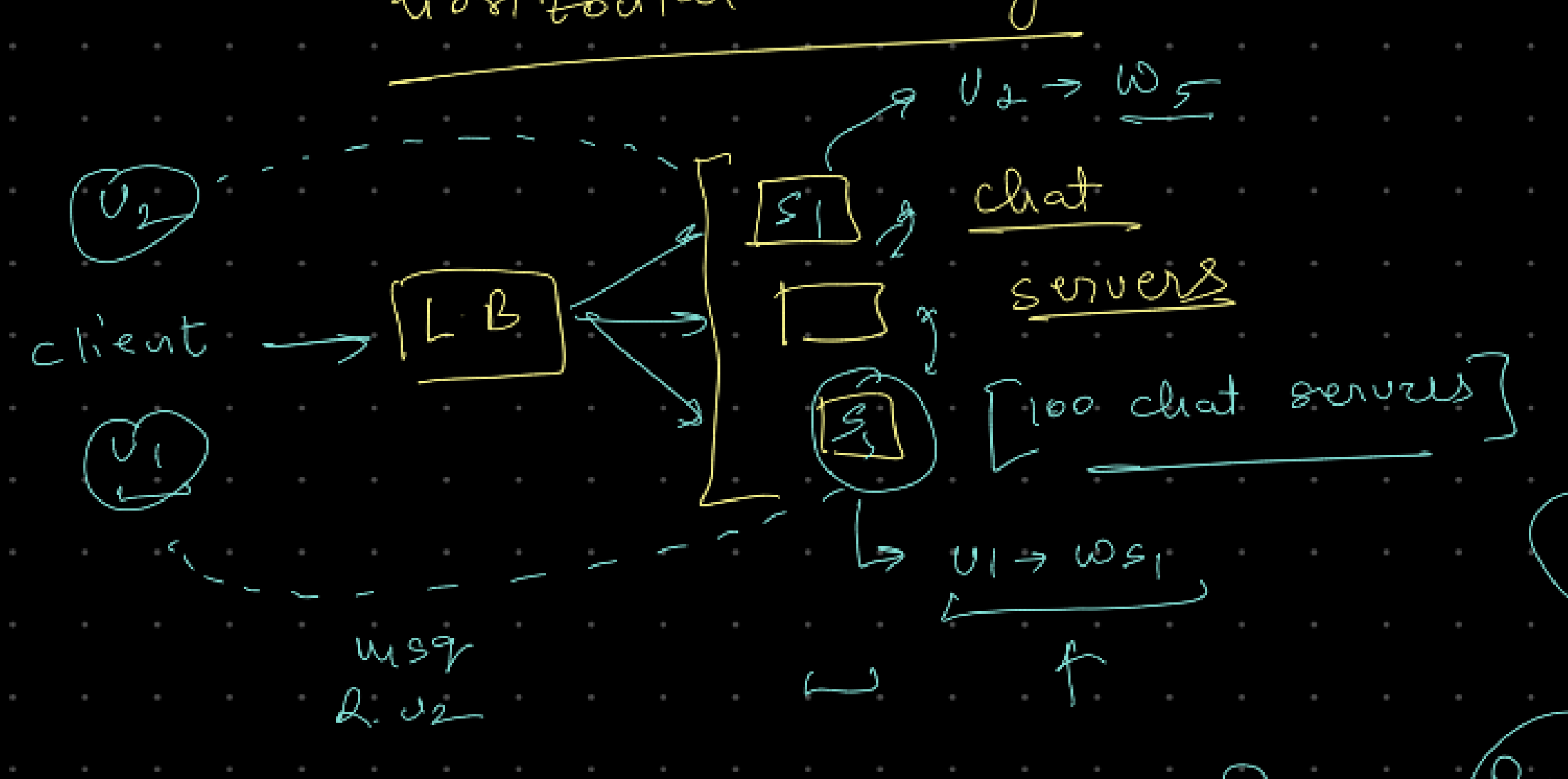

U1 → sends msg
R → U2
chat S
U2 → ~~??~~ w5
U2

[U1 → ws3
~~U2 → ws5~~]

websocket → Bi-directional
stable.

websocket connections ⇓ ⇑ {1 chat server} → (100M) connections.

horizontal scaling


U2 → w5
U2
client
LB
S1
S3
U1
chat servers
[100 chat servers]
↳ U1 → ws1
msg
R. U2

(C.S)
↳ U1 → ws1
↳ U2 → ws2

Gossiping

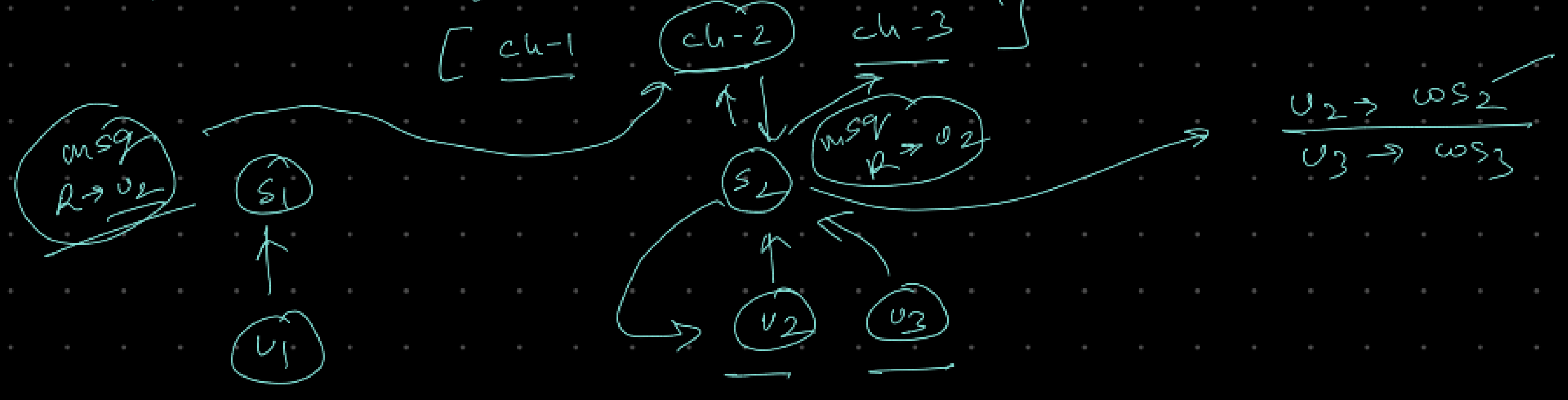
(U2 → ws2) (X)

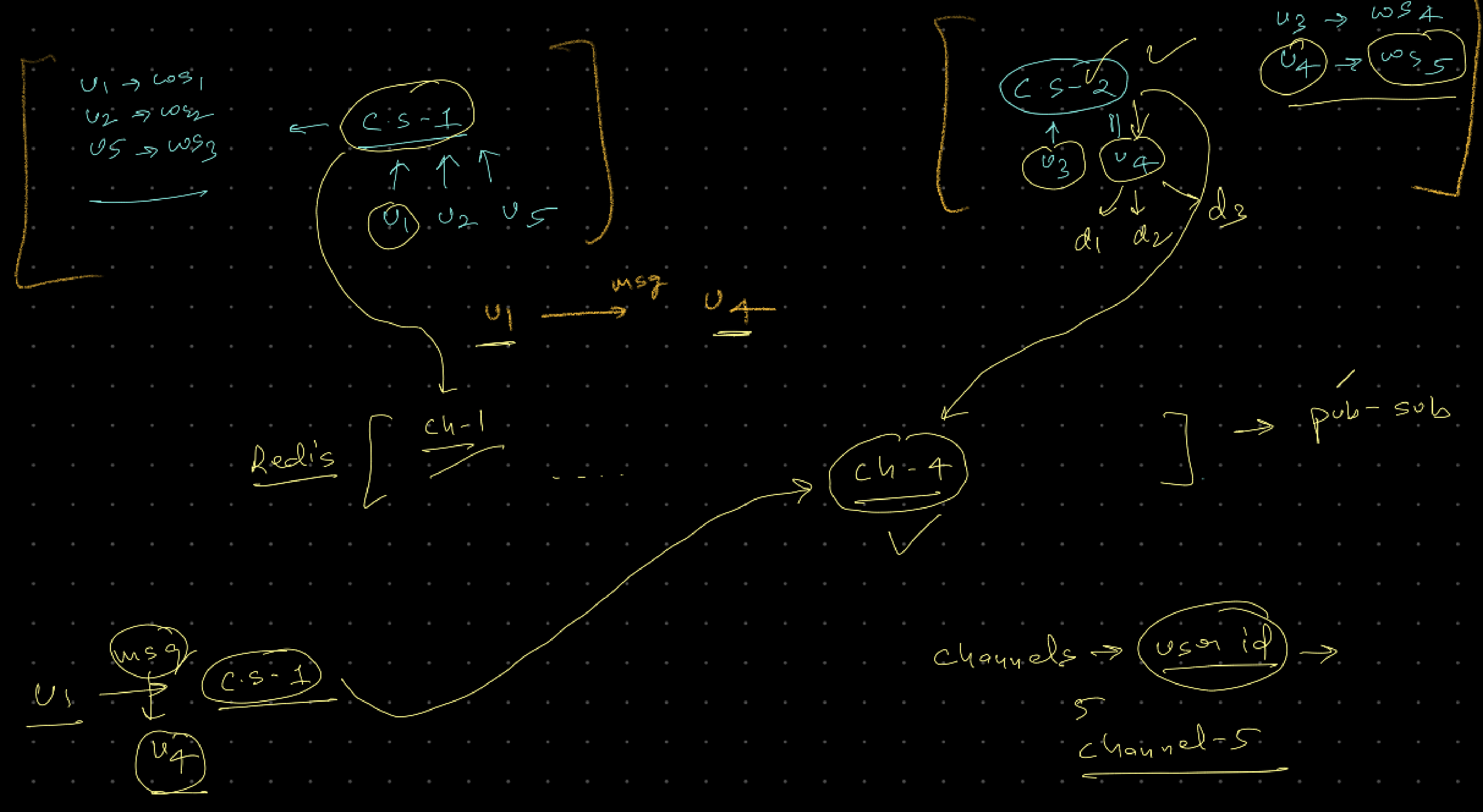
[In - memory cache] → (Redis) → in-memory

(Redis) → pub-sub model → queuing → kafka

U1 U2 U3

Redis → [1 [channel/queue] → per user]


[ch-1 ch-2 ch-3]
msg R → U2
S1
U1
S2
msg R → U2
U2
U3
U2 → ws2
U3 → ws3


U1 → ws1
U2 → ws2
U5 → ws3
C.S-1
U1 U2 U5
C.S-2
U3
U4
d1 d2 d3
U3 → ws4
U4 → ws5
U1 → msg → U4
Redis [ch-1 .... ch-4] → pub-sub
msg
U1
C.S-1
U4

channels → (user id) →
5
channel-5

send → receiver
↳ List [r1 r2 r3]

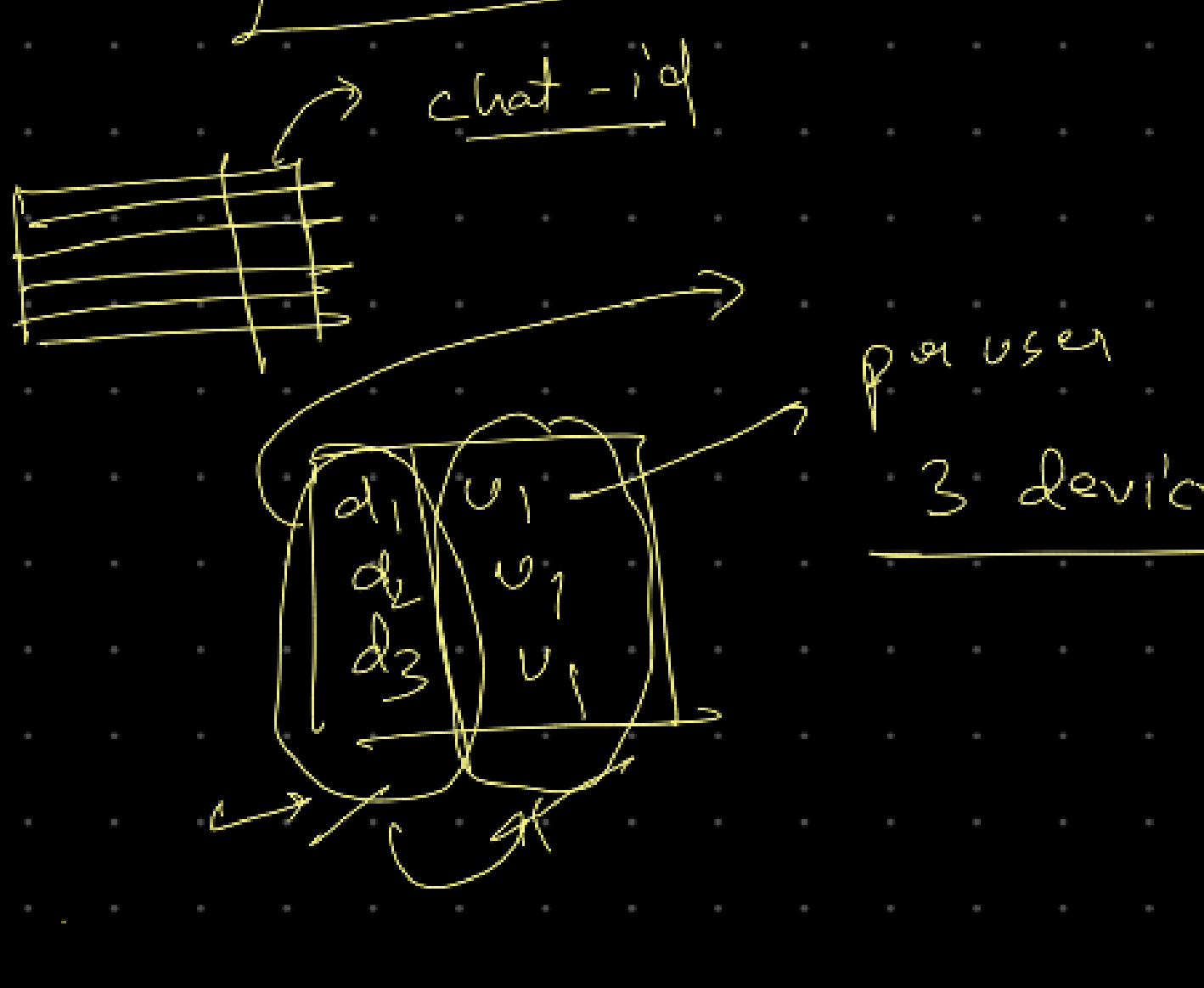
Chat-1
(msg)
R1 R2 R3

[ch-1
ch-2
ch-3]

[device → multiple] ✓ ✓

→ { user ✓
chat ✓
msg ✓ }
[group

Device → user
↳ device id ✓


chat-id
d1 d2 d3
U1 U1 U1
per user
3 devices